

Econ 202A Macroeconomics: Section 3

Kiyea Jin

November 5, 7, 2025

Section 3

1. Neoclassical Growth Model

- Upwind Scheme
- Boundary Conditions
- Sparse Matrix Routines

2. Huggett (1993) Model

- Model Overview
- Markov Chain Generator

Section 3-1:

Neoclassical Growth Model

Finite Difference Approximation

The finite difference approximations to HJB equation, associated with the FOC is:

$$\begin{aligned}\rho V_i &= U(c_i) + V'_i \cdot (k_i^\alpha - \delta k_i - c_i) \\ \text{with } c_i &= (U')^{-1}(V'_i)\end{aligned}\tag{1}$$

where $i = 1, \dots, I$, $V_i = V(k_i)$ with a uniform step size $\Delta k = k_{i+1} - k_i$.

Implicit Method: Matrix Representation

1. Define I discrete grid points for k , denoted as k_i for $i = 1, \dots, I$, and form an $I \times 1$ vector $\mathbf{k} = [k_1, k_2, \dots, k_I]'$.
2. Let $V_i = V(k_i)$. For each k_i on the grid, make an initial guess for the value function as an $I \times 1$ vector $\mathbf{V}^0 = [V_1^0, V_2^0, \dots, V_I^0]'$.

For iterations $n = 0, 1, 2, \dots$

3. Compute the derivative of the value function as an $I \times 1$ vector $(\mathbf{V}^n)'$ using an $I \times I$ finite-difference operator $\mathbf{D}$ such that $\mathbf{D}\mathbf{V}^n \simeq (\mathbf{V}^n)'$.
4. Compute the optimal consumption as an $I \times 1$ vector $\mathbf{c}^n$ from $\mathbf{c}^n = (U')^{-1}(\mathbf{D}\mathbf{V}^n)$.
5. Compute the optimal savings as an $I \times 1$ vector $\mathbf{s}^n$ from $\mathbf{s}^n = f(\mathbf{k}) - \delta\mathbf{k} - \mathbf{c}^n$.
6. Find $\mathbf{V}^{n+1}$ from:

$$\frac{1}{\Delta}(\mathbf{V}^{n+1} - \mathbf{V}^n) + \rho\mathbf{V}^{n+1} = U(\mathbf{c}^n) + (\mathbf{D}\mathbf{V}^{n+1}) \cdot \mathbf{s}^n$$

where the dot indicates element-wise multiplication.

7. If $\mathbf{V}^{n+1}$ is close enough to $\mathbf{V}^n$: stop. Otherwise, go to step 3.

Alternative matrix formulation:

$$\frac{1}{\Delta}(\mathbf{V}^{n+1} - \mathbf{V}^n) + \rho \mathbf{V}^{n+1} = U(\mathbf{c}^n) + \mathbf{S}^n \mathbf{D} \mathbf{V}^{n+1}$$

where $\mathbf{S}^n = \text{diag}(\mathbf{s}^n)$ is an $I \times I$ diagonal matrix with diagonals $\mathbf{s}^n = \{s_1^n, \dots, s_I^n\}$.

Equivalently, solve the linear system:

$$\mathbf{V}^{n+1} = \left(\left(\rho + \frac{1}{\Delta} \right) \mathbf{I} - \mathbf{S}^n \mathbf{D} \right)^{-1} \left[U(\mathbf{c}^n) + \frac{1}{\Delta} \mathbf{V}^n \right] \quad (2)$$

Key Challenges

1. Approximating the derivative of the value function, V'_i .
 - Mixed Method
 - **Upwind Scheme**
2. Solving the system, which is highly non-linear, requires iterative schemes.
 - Explicit Method
 - Implicit Method

The mixed method approximation for V'_i is defined as:

$$V'_i \simeq \begin{cases} V'_{i,F} = \frac{V_{i+1} - V_i}{\Delta k}, & i = 1 \\ V'_{i,C} = \frac{V_{i+1} - V_{i-1}}{2\Delta k}, & i \in \{2, 3, \dots, l-1\} \\ V'_{i,B} = \frac{V_i - V_{i-1}}{\Delta k}, & i = l \end{cases} \quad (3)$$

- Best finite difference approximation in this context: so-called “Upwind Scheme.”

- Best finite difference approximation in this context: so-called “Upwind Scheme.”
- Select the *differencing method* based on the *direction of the drift* of state variable:
 - Use **forward difference** whenever the drift of state variable is **positive**.
 - Use **backward difference** whenever the drift of state variable is **negative**.

Upwind Scheme in This Context

Compute optimal savings according to both the forward and backward difference approximations $V'_{i,F}$ and $V'_{i,B}$:

$$\begin{aligned}s_{i,F} &= k_i^\alpha - \delta k_i - (U')^{-1}(V'_{i,F}) \\ s_{i,B} &= k_i^\alpha - \delta k_i - (U')^{-1}(V'_{i,B})\end{aligned}\tag{4}$$

Upwind Scheme in This Context

Compute optimal savings according to both the forward and backward difference approximations $V'_{i,F}$ and $V'_{i,B}$:

$$\begin{aligned}s_{i,F} &= k_i^\alpha - \delta k_i - (U')^{-1}(V'_{i,F}) \\ s_{i,B} &= k_i^\alpha - \delta k_i - (U')^{-1}(V'_{i,B})\end{aligned}\tag{4}$$

Then, use the following approximation for V'_i , assuming the value function is concave:

$$V'_i = V'_{i,F} \mathbb{1}_{\{s_{i,F} > 0\}} + V'_{i,B} \mathbb{1}_{\{s_{i,B} < 0\}} + \bar{V}'_i \mathbb{1}_{\{s_{i,F} \leq 0 \leq s_{i,B}\}},\tag{5}$$

where $\mathbb{1}_{\{\cdot\}}$ denotes the indicator function, and $\bar{V}'_i = U'(c_i) = U'(k_i^\alpha - \delta k_i)$.

Why Upwind Scheme

- One might wonder why we would want to use such approximations, since centered approximations are more accurate.

Why Upwind Scheme

- One might wonder why we would want to use such approximations, since centered approximations are more accurate.
 $\implies$ The upwind scheme is preferred for its **stability** and alignment with the directional dynamics of the HJB equation.

Why Upwind Scheme

- One might wonder why we would want to use such approximations, since centered approximations are more accurate.
 $\implies$ The upwind scheme is preferred for its **stability** and alignment with the directional dynamics of the HJB equation.
- Intuitively, if saving is positive, what matters is how the value function changes when wealth increases by a small amount; and vice versa when saving is negative. (Achdou et al., 2022)

Consider the following one-dimensional linear advection equation:

$$\frac{\partial u}{\partial t} + a \frac{\partial u}{\partial x} = 0$$

which describes a wave propagating along the x -axis with a velocity a .

In the advection equation, there is an asymmetry due to directional translation at speed a . If $a > 0$, the solution moves to the right; if $a < 0$, it moves to the left. Recognizing this asymmetry, it is often optimal to use one-sided differences in the relevant direction (LeVeque, 2007).

- All our work so far applies only within the interior of the state space.

Boundary Conditions

- All our work so far applies only within the interior of the state space.
- How many boundary conditions are required?

Boundary Conditions

- All our work so far applies only within the interior of the state space.
- How many boundary conditions are required?
- Key intuition: Two boundary inequalities do same job as one boundary equality.

Boundary Conditions

- All our work so far applies only within the interior of the state space.
- How many boundary conditions are required?
- Key intuition: Two boundary inequalities do same job as one boundary equality.
- In practice, the stability of the algorithm improves by imposing a state constraint $k_{min} \leq k \leq k_{max}$, where k_{min} and k_{max} represent the lower and upper bounds of the state space used in computations.

Boundary Conditions

We aim to keep households within the state space, so we enforce that they neither borrow as $k \rightarrow k_{min}$ nor save as $k \rightarrow k_{max}$.

Boundary Conditions

We aim to keep households within the state space, so we enforce that they neither borrow as $k \rightarrow k_{min}$ nor save as $k \rightarrow k_{max}$.

The constraint $k \geq k_{min}$ is enforced by setting

$$V'_{1,B} = U'(f(k_1) - \delta k_1) \quad (6)$$

The state constraint is applied whenever the forward difference approximation would yield negative savings, $s_{1,F} \leq 0$. If $s_{1,F} > 0$, the forward difference approximation $V'_{1,F}$ is used at the boundary, implying that the value function “never sees the state constraint.”

Boundary Conditions

We aim to keep households within the state space, so we enforce that they neither borrow as $k \rightarrow k_{min}$ nor save as $k \rightarrow k_{max}$.

The constraint $k \geq k_{min}$ is enforced by setting

$$V'_{1,B} = U'(f(k_1) - \delta k_1) \quad (6)$$

The state constraint is applied whenever the forward difference approximation would yield negative savings, $s_{1,F} \leq 0$. If $s_{1,F} > 0$, the forward difference approximation $V'_{1,F}$ is used at the boundary, implying that the value function “never sees the state constraint.”

The constraint $k \leq k_{max}$ is enforced by setting

$$V'_{I,F} = U'(f(k_I) - \delta k_I) \quad (7)$$

Implicit Method Algorithm

1. Construct I discrete grid points for k , denoted as k_i where $i = 1, \dots, I$, and let $V_i = V(k_i)$.
2. For each k_i on the grid, guess for a value of $\mathbf{V}^0 = (V_1^0, V_2^0 \dots V_I^0)$.

For iterations $n = 0, 1, 2, \dots$,

3. Compute $(V_i^n)'$ using (4), (5), (6), and (7).
4. Compute $\mathbf{c}^n$ from $c_i^n = (U')^{-1}[(V_i^n)']$.
5. Find V_i^{n+1} from (2).
6. If $\mathbf{V}^{n+1}$ is close enough to $\mathbf{V}^n$: stop. Otherwise, go to step 3.

Exercise: Numerical Solution of the Neoclassical Growth Model

In this exercise, apply the finite difference method with an **upwind scheme** to numerically solve the Hamilton-Jacobi-Bellman (HJB) equation for the Neoclassical Growth Model:

$$\rho V(k) = \max_c \{U(c) + V'(k) \cdot (f(k) - \delta k - c)\} \quad (8)$$

In this exercise, we also consider **boundary conditions** for the value function $V(k)$.

Implicit Method: Matrix Representation

1. Define I discrete grid points for k , denoted as k_i for $i = 1, \dots, I$, and form an $I \times 1$ vector $\mathbf{k} = [k_1, k_2, \dots, k_I]'$.
2. Let $V_i = V(k_i)$. For each k_i on the grid, make an initial guess for the value function as an $I \times 1$ vector $\mathbf{V}^0 = [V_1^0, V_2^0, \dots, V_I^0]'$.
3. Construct the $I \times I$ forward and backward difference matrix operators $\mathbf{D}_F$ and $\mathbf{D}_B$ such that $\mathbf{D}_F \mathbf{V}^n \simeq (\mathbf{V}^n)'_F$ and $\mathbf{D}_B \mathbf{V}^n \simeq (\mathbf{V}^n)'_B$.

Implicit Method: Matrix Representation

For iterations $n = 0, 1, 2, \dots$

4. Compute the derivative of the value function as an $I \times 1$ vector using both forward and backward difference operators:

$$(\mathbf{V}_F^n)' = \mathbf{D}_F \mathbf{V}^n \qquad (\mathbf{V}_B^n)' = \mathbf{D}_B \mathbf{V}^n$$

5. Set the first element of $(\mathbf{V}_B^n)'$ as:

$$(V_{1,B}^n)' = U'(f(k_1) - \delta k_1)$$

and the last element of $(\mathbf{V}_F^n)'$ as:

$$(V_{I,F}^n)' = U'(f(k_I) - \delta k_I)$$

Implicit Method: Matrix Representation

6. Compute the optimal consumption as an $l \times 1$ vector:

$$\mathbf{c}_F^n = (U')^{-1}[(\mathbf{V}_F^n)'] \quad \mathbf{c}_B^n = (U')^{-1}[(\mathbf{V}_B^n)']$$

7. Calculate the optimal savings as an $l \times 1$ vector:

$$\mathbf{s}_F^n = f(\mathbf{k}) - \delta \mathbf{k} - \mathbf{c}_F^n \quad \mathbf{s}_B^n = f(\mathbf{k}) - \delta \mathbf{k} - \mathbf{c}_B^n$$

8. Create indicator vectors:

$$\mathbf{l}_F^n = [l_{1,F}^n, l_{2,F}^n, \dots, l_{l,F}^n]' \quad \mathbf{l}_B^n = [l_{1,B}^n, l_{2,B}^n, \dots, l_{l,B}^n]'$$

where $l_{i,F}^n = 1$ if $s_{i,F}^n > 0$ and $l_{i,B}^n = 1$ if $s_{i,B}^n < 0$ for $i = 1, \dots, l$.

Implicit Method: Matrix Representation

9. Compute optimal consumption as:

$$\mathbf{c}^n = \mathbf{I}_F^n \cdot \mathbf{c}_F^n + \mathbf{I}_B^n \cdot \mathbf{c}_B^n + (1 - \mathbf{I}_F^n - \mathbf{I}_B^n) \cdot \mathbf{c}_0^n$$

where the dot indicates element-wise multiplication.

10. Compute optimal savings as:

$$\mathbf{s}^n = \mathbf{I}_F^n \cdot \mathbf{s}_F^n + \mathbf{I}_B^n \cdot \mathbf{s}_B^n$$

where the dot indicates element-wise multiplication.

11. Construct an $I \times I$ matrix:

$$\mathbf{S}^n \mathbf{D} = \text{diag}(\mathbf{I}_F^n \cdot \mathbf{s}_F^n) \mathbf{D}_F + \text{diag}(\mathbf{I}_B^n \cdot \mathbf{s}_B^n) \mathbf{D}_B$$

Implicit Method: Matrix Representation

12. Find $\mathbf{V}^{n+1}$ from:

$$\mathbf{V}^{n+1} = \left[\left(\rho + \frac{1}{\Delta} \right) \mathbf{I} - \mathbf{S}^n \mathbf{D} \right]^{-1} \left[U(\mathbf{c}^n) + \frac{1}{\Delta} \mathbf{V}^n \right] \quad (9)$$

13. If $\mathbf{V}^{n+1}$ is close enough to $\mathbf{V}^n$, stop. Otherwise, go to step 4.

- The matrices $\mathbf{D}$, $\mathbf{I}$, and $\mathbf{S}^n$ are highly sparse, meaning they contain a large number of zero elements.
- Sparse matrices can be manipulated efficiently in MATLAB by storing only the non-zero elements, which significantly reduces memory usage and speeds up computations through optimized algorithms that skip operations on zero elements.

Useful Sparse Matrix Functions

- The `spy` function in MATLAB is useful for checking the sparsity pattern of matrices, displaying non-zero elements as dots. This visual verification ensures matrices are constructed as intended.

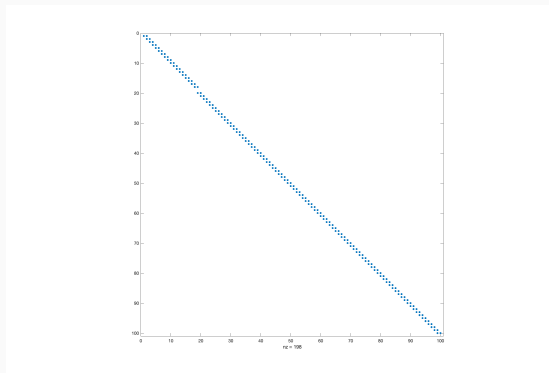


Figure 1: Visualization of Matrix SD

Useful Sparse Matrix Functions

- `sparse`: Creates a sparse matrix by storing only the non-zero elements.

sparse Create sparse matrix.

`S = sparse(X)` converts a sparse or full matrix to sparse form by squeezing out any zero elements.

- `speye`: Generates a sparse identity matrix.

speye Sparse identity matrix.

`speye(M,N)` and `speye([M N])` form an M-by-N sparse matrix with 1's on the main diagonal. `speye(N)` abbreviates `speye(N,N)`.

- `spdiags`: Constructs sparse matrices with specified diagonals.

`A = spdiags(B,d,m,n)` creates an m-by-n sparse matrix from the columns of B and places them along the diagonals specified by d.

Section 3-2: Huggett Model

- We solve a continuous-time version of Huggett (1993), which is one of the simplest heterogeneous agent models that captures key features of richer models.
- The model studies agents in incomplete markets facing (i) uninsurable idiosyncratic income risk and (ii) borrowing constraints.

- The model features an exchange economy with a continuum of agents, where the total mass of agents equals one.
- Each period, households experience uninsurable idiosyncratic income shocks and can be either employed or unemployed.
- Each agent's income follows a Markov process: an employed individual becomes unemployed with probability $\lambda_e = \text{Prob}(z_{t+1} = z_u \mid z_t = z_e)$, and an unemployed individual becomes employed with probability $\lambda_u = \text{Prob}(z_{t+1} = z_e \mid z_t = z_u)$.
- When employed, they receive income $z_e = w(1 - \tau)$; when unemployed, they receive unemployment benefits $z_u = \mu w$, with $z_e > z_u$.
- They cannot borrow beyond a certain limit.
- The only asset available is a risk-free bond, whose interest rate is determined in equilibrium. For partial-equilibrium analysis, prices are assumed to be constant and exogenously given.

Huggett Model in Discrete-Time Recursive Formulation

We start with the following Bellman equations for employed and unemployed households:

$$\begin{aligned} V_e(a_t) &= \max_{c_t, a_{t+1}} \{ U(c_t) + (1 - \rho)[(1 - \lambda_e)V_e(a_{t+1}) + \lambda_e V_u(a_{t+1})] \} \\ \text{s.t. } c_t + a_{t+1} &= z_e + (1 + r_t)a_t \\ a_t &\geq \underline{a} \quad \forall t \end{aligned} \tag{10}$$

$$\begin{aligned} V_u(a_t) &= \max_{c_t, a_{t+1}} \{ U(c_t) + (1 - \rho)[(1 - \lambda_u)V_u(a_{t+1}) + \lambda_u V_e(a_{t+1})] \} \\ \text{s.t. } c_t + a_{t+1} &= z_u + (1 + r_t)a_t \\ a_t &\geq \underline{a} \quad \forall t \end{aligned} \tag{11}$$

Discrete to Continuous-Time Transformation

Over a time interval of Δ units, the model can be expressed as:

$$\begin{aligned} V_j(a_t) &= \max_{c_t, a_{t+\Delta}} \left\{ \Delta U(c_t) + (1 - \Delta\rho) \left[(1 - \Delta\lambda_j) V_j(a_{t+\Delta}) + \Delta\lambda_j V_u(a_{t+\Delta}) \right] \right\} \\ \text{s.t. } \quad &\Delta c_t + a_{t+\Delta} = \Delta z_j + (1 + \Delta r_t) a_t \\ &a_t \geq \underline{a} \quad \forall t \end{aligned}$$

where $j \in \{e, u\}$ represents employment states (employed e and unemployed u).

Discrete to Continuous-Time Transformation

Subtracting $V(a_t)$ from both sides and substituting the constraints into $V(a_{t+\Delta})$, we get:

$$\begin{aligned} 0 = \max_{c_t} \{ & \Delta U(c_t) + [V_j(a_t + \Delta(z_j + r_t a_t - c_t)) - V_j(a_t)] \\ & - \Delta(\rho + \lambda_j - \Delta\rho\lambda_j)V_j(a_t + \Delta(z_j + r_t a_t - c_t)) \\ & + (1 - \Delta\rho)\Delta\lambda_j V_u(a_t + \Delta(z_j + r_t a_t - c_t)) \} \end{aligned}$$

Dividing both sides by Δ :

$$\begin{aligned} 0 = \max_{c_t} \{ & U(c_t) + \left[\frac{V_j(a_t + \Delta(z_j + r_t a_t - c_t)) - V_j(a_t)}{\Delta} \right] \\ & - (\rho + \lambda_j - \Delta\rho\lambda_j)V_j(a_t + \Delta(z_j + r_t a_t - c_t)) \\ & + (1 - \Delta\rho)\lambda_j V_u(a_t + \Delta(z_j + r_t a_t - c_t)) \} \end{aligned}$$

Taking the limit as $\Delta \rightarrow 0$, we obtain:

$$0 = \max_{c_t} \{ U(c_t) + V'_j(a_t)(z_j + r_t a_t - c_t) - \rho V_j(a_t) + \lambda_j(V_u(a_t) - V_j(a_t)) \}$$

Hamilton-Jacobi-Bellman (HJB) Equation

Rearranging terms and dropping time notation leads to the HJB equation:

$$\begin{aligned}\rho V_e(a) &= \max_c \{ U(c) + V'_e(a)(z_e + ra - c) + \lambda_e(V_u(a) - V_e(a)) \} \\ \rho V_u(a) &= \max_c \{ U(c) + V'_u(a)(z_u + ra - c) + \lambda_u(V_e(a) - V_u(a)) \}\end{aligned}\tag{12}$$

Hamilton-Jacobi-Bellman (HJB) Equation

Rearranging terms and dropping time notation leads to the HJB equation:

$$\begin{aligned}\rho V_e(a) &= \max_c \{ U(c) + V'_e(a)(z_e + ra - c) + \lambda_e(V_u(a) - V_e(a)) \} \\ \rho V_u(a) &= \max_c \{ U(c) + V'_u(a)(z_u + ra - c) + \lambda_u(V_e(a) - V_u(a)) \}\end{aligned}\tag{12}$$

The optimal consumption, $c_j = c_j(a)$, is derived from the first-order condition:

$$U'(c_j) = V'_j(a)\tag{13}$$

Hamilton-Jacobi-Bellman (HJB) Equation

Rearranging terms and dropping time notation leads to the HJB equation:

$$\begin{aligned}\rho V_e(a) &= \max_c \{ U(c) + V'_e(a)(z_e + ra - c) + \lambda_e(V_u(a) - V_e(a)) \} \\ \rho V_u(a) &= \max_c \{ U(c) + V'_u(a)(z_u + ra - c) + \lambda_u(V_e(a) - V_u(a)) \}\end{aligned}\tag{12}$$

The optimal consumption, $c_j = c_j(a)$, is derived from the first-order condition:

$$U'(c_j) = V'_j(a)\tag{13}$$

Denote $s_j(a) = z_j + ra - c_j$, which represents optimal savings.

Hamilton-Jacobi-Bellman (HJB) Equation

Rearranging terms and dropping time notation leads to the HJB equation:

$$\begin{aligned}\rho V_e(a) &= \max_c \{ U(c) + V'_e(a)(z_e + ra - c) + \lambda_e(V_u(a) - V_e(a)) \} \\ \rho V_u(a) &= \max_c \{ U(c) + V'_u(a)(z_u + ra - c) + \lambda_u(V_e(a) - V_u(a)) \}\end{aligned}\tag{12}$$

The optimal consumption, $c_j = c_j(a)$, is derived from the first-order condition:

$$U'(c_j) = V'_j(a)\tag{13}$$

Denote $s_j(a) = z_j + ra - c_j$, which represents optimal savings.

The state constraint $a \geq \underline{a}$ motivates a **boundary condition**:

$$V'_j(\underline{a}) \geq U'(z_j + r\underline{a})\tag{14}$$

Generator of a Markov Process

- The (*infinitesimal*) generator $\mathcal{A}$ maps a function $f(t, x)$ to the **instantaneous expected rate of change** of $f(t, X_t)$.

$$(\mathcal{A}f)(t, x) = \lim_{\Delta t \downarrow 0} \frac{\mathbb{E}[f(t + \Delta t, X(t + \Delta t)) - f(t, x) \mid X(t) = x]}{\Delta t}.$$

- Intuition: look an instant ahead, average the increment in f over the randomness of X , then scale by the step length.

Consider the Itô diffusion

$$dX_t = \mu(X_t, t) dt + \sigma(X_t, t) dW_t,$$

with $f \in C^{1,2}$.

Using Itô's formula, the generator is

$$(\mathcal{A}f)(t, x) = f_t(t, x) + \mu(x, t) f_x(t, x) + \frac{1}{2} \sigma^2(x, t) f_{xx}(t, x).$$

Generator of Markov Chain in This Context

In this model, each agent's endowment follows a Markov chain, where individual i experiences earnings (employment) shocks and transitions between states $z_t \in \{z_e, z_u\}$ — employed (z_e) and unemployed (z_u) — with transition rates λ_e and λ_u , respectively.

Generator of Markov Chain in This Context

In this model, each agent's endowment follows a Markov chain, where individual i experiences earnings (employment) shocks and transitions between states $z_t \in \{z_e, z_u\}$ — employed (z_e) and unemployed (z_u) — with transition rates λ_e and λ_u , respectively.

The associated generator is:

$$\mathcal{A}^z = \begin{pmatrix} -\lambda_e & \lambda_e \\ \lambda_u & -\lambda_u \end{pmatrix}.$$

This matrix represents the transitions between states. Households transition out of state j at rate λ_j , where $j \in \{e, u\}$.

Generator of Markov Chain in This Context

In this model, each agent's endowment follows a Markov chain, where individual i experiences earnings (employment) shocks and transitions between states $z_t \in \{z_e, z_u\}$ — employed (z_e) and unemployed (z_u) — with transition rates λ_e and λ_u , respectively.

The associated generator is:

$$\mathcal{A}^z = \begin{pmatrix} -\lambda_e & \lambda_e \\ \lambda_u & -\lambda_u \end{pmatrix}.$$

This matrix represents the transitions between states. Households transition out of state j at rate λ_j , where $j \in \{e, u\}$.

The rows of the matrix sum to 0, reflecting a conservation of mass in this continuous-time setting. Unlike discrete-time Markov chains, where row sums are 1 (representing probabilities), here, the row sums represent net transition rates.

The finite difference approximation to the two HJB equations (12) and (13) is:

$$\begin{aligned} \rho V_{i,j} &= U(c_{i,j}) + V'_{i,j} \cdot (z_j + ra_i - c_{i,j}) + \lambda_j (V_{i,-j} - V_{i,j}), \quad j = e, u \\ \text{with } c_{i,j} &= (U')^{-1}(V'_{i,j}) \end{aligned} \tag{15}$$

where $i = 1, \dots, I$ and $j \in \{e, u\}$, $V_{i,j} = V_j(a_i)$ with a uniform step size $\Delta a = a_{i+1} - a_i$.

1. Approximating the **derivative** of the value function, V'_i .
 - Mixed Method
 - **Upwind Scheme**
2. Solving the system, which is highly **non-linear**, requires iterative schemes.
 - Explicit Method
 - **Implicit Method**

Compute optimal savings according to both the forward and backward difference approximations $V'_{i,F}$ and $V'_{i,B}$:

$$\begin{aligned}s_{i,j,F} &= z_j + ra_i - (U')^{-1}(V'_{i,j,F}) \\ s_{i,j,B} &= z_j + ra_i - (U')^{-1}(V'_{i,j,B})\end{aligned}\tag{16}$$

Then, use the following approximation for $V'_{i,j}$:

$$V'_{i,j} = V'_{i,j,F} \mathbb{1}_{\{s_{i,j,F} > 0\}} + V'_{i,j,B} \mathbb{1}_{\{s_{i,j,B} < 0\}} + \bar{V}'_i \mathbb{1}_{\{s_{i,j,F} \leq 0 \leq s_{i,j,B}\}}\tag{17}$$

where $\mathbb{1}_{\{\cdot\}}$ denotes the indicator function, and $\bar{V}'_i = U'(c_{i,j}) = U'(z_j + ra_i)$.

V^{n+1} is *implicitly* defined by:

$$\frac{V_{i,j}^{n+1} - V_{i,j}^n}{\Delta} + \rho V_{i,j}^{n+1} = U(c_{i,j}^n) + (V_{i,j}^{n+1})' \cdot (z_j - \delta a_i - c_{i,j}^n) + \lambda_j (V_{i,-j}^{n+1} - V_{i,j}^{n+1}) \quad (18)$$

The step size Δ can be arbitrarily large. (Achdou et al., 2022)

Upwind Scheme with Implicit Method

We use the following finite difference approximation:

$$\frac{V_{i,j}^{n+1} - V_{i,j}^n}{\Delta} + \rho V_{i,j}^{n+1} = U(c_{i,j}^n) + (V_{i,j,F}^{n+1})' \cdot [z_j + ra_i - c_{i,j,F}^n]^+ + (V_{i,j,B}^{n+1})' \cdot [z_j + ra_i - c_{i,j,B}^n]^- + \lambda_j (V_{i,-j}^{n+1} - V_{i,j}^{n+1}) \quad (19)$$

with $c_{i,j} = (U')^{-1}(V'_{i,j})$.

where for any number x , $x^+ = \max\{x, 0\}$ and $x^- = \min\{x, 0\}$

Upwind Scheme with Implicit Method

Equivalently:

$$\frac{V_{i,j}^{n+1} - V_{i,j}^n}{\Delta} + \rho V_{i,j}^{n+1} = U(c_{i,j}^n) + \frac{V_{i+1,j}^{n+1} - V_{i,j}^{n+1}}{\Delta a} (s_{i,j,F}^n)^+ + \frac{V_{i,j}^{n+1} - V_{i-1,j}^{n+1}}{\Delta a} (s_{i,j,B}^n)^- + \lambda_j (V_{i,-j}^{n+1} - V_{i,j}^{n+1}) \quad (20)$$

Upwind Scheme with Implicit Method

Collecting terms with the same subscripts on the right-hand side:

$$\frac{V_{i,j}^{n+1} - V_{i,j}^n}{\Delta} + \rho V_{i,j}^{n+1} = u(c_{i,j}^n) + V_{i-1,j}^{n+1} x_{i,j} + V_{i,j}^{n+1} y_{i,j} + V_{i+1,j}^{n+1} z_{i,j} + V_{i,-j}^{n+1} \lambda_j \quad (21)$$

where

$$\begin{aligned} x_{i,j} &= -\frac{(s_{i,j,B}^n)^-}{\Delta a} \\ y_{i,j} &= -\frac{(s_{i,j,F}^n)^+}{\Delta a} + \frac{(s_{i,j,B}^n)^-}{\Delta a} - \lambda_j \\ z_{i,j} &= \frac{(s_{i,j,F}^n)^+}{\Delta a} \end{aligned} \quad (22)$$

Upwind Scheme with Implicit Method

Equation (21) with (22) represents a system of $2 \times I$ linear equations, which can be written in matrix notation as:

$$\frac{1}{\Delta}(\mathbf{V}^{n+1} - \mathbf{V}^n) + \rho \mathbf{V}^{n+1} = U(\mathbf{c}^n) + \mathbf{P}^n \mathbf{V}^{n+1}$$

where

$$\mathbf{P}^n = \left(\begin{array}{ccccc|ccccc} y_{1,1} & z_{1,1} & 0 & \dots & 0 & \lambda_1 & 0 & 0 & \dots & 0 \\ x_{2,1} & y_{2,1} & z_{2,1} & 0 & \dots & 0 & \lambda_1 & 0 & 0 & \dots \\ 0 & x_{3,1} & y_{3,1} & z_{3,1} & 0 & 0 & 0 & \lambda_1 & 0 & \dots \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots & 0 \\ 0 & \ddots & \ddots & x_{I,1} & y_{I,1} & 0 & 0 & 0 & 0 & \lambda_1 \\ \hline \lambda_2 & 0 & 0 & 0 & 0 & y_{1,2} & z_{1,2} & 0 & 0 & 0 \\ 0 & \lambda_2 & 0 & 0 & 0 & x_{2,2} & y_{2,2} & z_{2,2} & 0 & 0 \\ 0 & 0 & \lambda_2 & 0 & 0 & 0 & x_{3,2} & y_{3,2} & z_{3,2} & 0 \\ 0 & \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots & 0 \\ 0 & \dots & \dots & 0 & \lambda_2 & 0 & \dots & 0 & x_{I,2} & y_{I,2} \end{array} \right) \left. \begin{array}{l} \vphantom{\begin{matrix} y_{1,1} \\ x_{2,1} \\ 0 \\ \vdots \\ 0 \end{matrix}} \right\} I \text{ elements} \\ \left. \begin{array}{l} \vphantom{\begin{matrix} \lambda_2 \\ 0 \\ 0 \\ 0 \end{matrix}} \end{array} \right\} I \text{ elements}$$

The borrowing constraint $a_t \geq \underline{a}$ is enforced by setting:

$$V'_{1,j,B} = U'(z_j + r\underline{a}) \quad (23)$$

In practice, the stability of the algorithm improves by imposing a state constraint $a \leq a_{max}$ where a_{max} is the upper bounds of the state space used in computations. This can be achieved by setting:

$$V'_{l,j,F} = U'(z_j + ra_l) \quad (24)$$

Initial Guess for the Value Function

A natural initial guess is the value function of “staying put”:

$$V_{i,j}^0 = \frac{U(z_j + ra_i)}{\rho} \quad (25)$$

Implicit Method: Matrix Representation

1. Define I discrete grid points for a , denoted as a_i for $i = 1, \dots, I$, and form an $I \times 1$ vector $\mathbf{a} = [a_1, a_2, \dots, a_I]'$.
2. Let $V_{i,j} = V_j(a_i)$. For each a_i on the grid, make an initial guess for the value function as two $I \times 1$ vectors $\mathbf{V}_e^0 = [V_{1,e}^0, V_{2,e}^0, \dots, V_{I,e}^0]'$ and $\mathbf{V}_u^0 = [V_{1,u}^0, V_{2,u}^0, \dots, V_{I,u}^0]'$.
3. Construct the stacked $2I \times 1$ vector $\mathbf{V}^0 = [\mathbf{V}_e^0, \mathbf{V}_u^0]'$.
4. Construct the $I \times I$ forward and backward difference matrix operators $\mathbf{D}_F$ and $\mathbf{D}_B$ such that $\mathbf{D}_F \mathbf{V}^n \simeq (\mathbf{V}^n)'_F$ and $\mathbf{D}_B \mathbf{V}^n \simeq (\mathbf{V}^n)'_B$.

Implicit Method: Matrix Representation

5. Construct a $2I \times 2I$ matrix as follows:

$$\mathbf{A} = \left(\begin{array}{cccc|cccc} -\lambda_e & 0 & \cdots & 0 & \lambda_e & 0 & \cdots & 0 \\ 0 & -\lambda_e & 0 & 0 & 0 & \lambda_e & 0 & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots & \vdots & \ddots & \ddots \\ 0 & 0 & 0 & -\lambda_e & 0 & 0 & 0 & \lambda_e \\ \hline \lambda_u & 0 & \cdots & 0 & -\lambda_u & 0 & \cdots & 0 \\ 0 & \lambda_u & 0 & 0 & 0 & -\lambda_u & 0 & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots & \vdots & \ddots & \ddots \\ 0 & 0 & 0 & \lambda_u & 0 & 0 & 0 & -\lambda_u \end{array} \right) \left. \begin{array}{l} \vphantom{\begin{pmatrix} \\ \\ \\ \end{pmatrix}} \right\} I \text{ elements} \\ \vphantom{\begin{pmatrix} \\ \\ \\ \end{pmatrix}} \right\} I \text{ elements}$$

Implicit Method: Matrix Representation

For iterations $n = 0, 1, 2, \dots$

6. Compute the derivative of the value function as an $I \times 1$ vector using both forward and backward difference matrix operators:

$$(\mathbf{V}_{e,F}^n)' = \mathbf{D}_F \mathbf{V}_e^n$$

$$(\mathbf{V}_{u,F}^n)' = \mathbf{D}_F \mathbf{V}_u^n$$

$$(\mathbf{V}_{e,B}^n)' = \mathbf{D}_B \mathbf{V}_e^n$$

$$(\mathbf{V}_{u,B}^n)' = \mathbf{D}_B \mathbf{V}_u^n$$

7. Set the first elements of $\mathbf{V}_{j,B}^n$ as:

$$(V_{1,e,B}^n)' = U'(z_e + r\underline{a})$$

$$(V_{1,u,B}^n)' = U'(z_u + r\underline{a})$$

and the last elements of $\mathbf{V}_{j,F}^n$ as:

$$(V_{I,e,F}^n)' = U'(z_e + r\bar{a})$$

$$(V_{I,u,F}^n)' = U'(z_u + r\bar{a})$$

Implicit Method: Matrix Representation

9. Compute the optimal consumption as an $I \times 1$ vector from:

$$\mathbf{c}_{e,F}^n = (U')^{-1}[(\mathbf{V}_{e,F}^n)']$$

$$\mathbf{c}_{u,F}^n = (U')^{-1}[(\mathbf{V}_{u,F}^n)']$$

$$\mathbf{c}_{e,B}^n = (U')^{-1}[(\mathbf{V}_{e,B}^n)']$$

$$\mathbf{c}_{u,B}^n = (U')^{-1}[(\mathbf{V}_{u,B}^n)']$$

10. Calculate the optimal savings as an $I \times 1$ vector from:

$$\mathbf{s}_{e,F}^n = z_e + r\mathbf{a} - \mathbf{c}_{e,F}^n$$

$$\mathbf{s}_{u,F}^n = z_u + r\mathbf{a} - \mathbf{c}_{u,F}^n$$

$$\mathbf{s}_{e,B}^n = z_e + r\mathbf{a} - \mathbf{c}_{e,B}^n$$

$$\mathbf{s}_{u,B}^n = z_u + r\mathbf{a} - \mathbf{c}_{u,B}^n$$

11. Create indicator vectors:

$$\mathbf{l}_{e,F}^n = [l_{1,e,F}^n, l_{2,e,F}^n, \dots, l_{I,e,F}^n]'$$

$$\mathbf{l}_{u,F}^n = [l_{1,u,F}^n, l_{2,u,F}^n, \dots, l_{I,u,F}^n]'$$

$$\mathbf{l}_{e,B}^n = [l_{1,e,B}^n, l_{2,e,B}^n, \dots, l_{I,e,B}^n]'$$

$$\mathbf{l}_{u,B}^n = [l_{1,u,B}^n, l_{2,u,B}^n, \dots, l_{I,u,B}^n]'$$

where $l_{i,j,F}^n = 1$ if $s_{i,j,F}^n > 0$ and $l_{i,j,B}^n = 1$ if $s_{i,j,B}^n < 0$ for $i = 1, \dots, I$ and $j = e, u$.

Implicit Method: Matrix Representation

12. Compute optimal consumption as follows:

$$\begin{aligned}c_e^n &= l_{e,F}^n \cdot c_{e,F}^n + l_{e,B}^n \cdot c_{e,B}^n + (1 - l_{e,F}^n - l_{e,B}^n) \cdot c_{e,0}^n \\c_u^n &= l_{u,F}^n \cdot c_{u,F}^n + l_{u,B}^n \cdot c_{u,B}^n + (1 - l_{u,F}^n - l_{u,B}^n) \cdot c_{u,0}^n\end{aligned}$$

13. Compute optimal savings as follows:

$$\begin{aligned}s_e^n &= l_{e,F}^n \cdot s_{e,F}^n + l_{e,B}^n \cdot s_{e,B}^n \\s_u^n &= l_{u,F}^n \cdot s_{u,F}^n + l_{u,B}^n \cdot s_{u,B}^n\end{aligned}$$

14. Construct two $I \times I$ diagonal matrices as follows:

$$\begin{aligned}S_e^n D_e^n &= \text{diag}(l_{e,F}^n \cdot s_{e,F}^n) D_F + \text{diag}(l_{e,B}^n \cdot s_{e,B}^n) D_B \\S_u^n D_u^n &= \text{diag}(l_{u,F}^n \cdot s_{u,F}^n) D_F + \text{diag}(l_{u,B}^n \cdot s_{u,B}^n) D_B\end{aligned}$$

Implicit Method: Matrix Representation

15. Construct a $2I \times 2I$ matrix $\mathbf{S}^n \mathbf{D}^n$ as follows:

$$\mathbf{S}^n \mathbf{D}^n = \begin{pmatrix} \mathbf{S}_e^n \mathbf{D}_e^n & 0 \\ 0 & \mathbf{S}_u^n \mathbf{D}_u^n \end{pmatrix} \quad (26)$$

16. Construct the matrix $\mathbf{P}^n$ as $\mathbf{P}^n = \mathbf{S}^n \mathbf{D}^n + \mathbf{A}$

17. Find $\mathbf{V}^{n+1}$ from:

$$\frac{1}{\Delta}(\mathbf{V}^{n+1} - \mathbf{V}^n) + \rho \mathbf{V}^{n+1} = U(\mathbf{c}^n) + \mathbf{P}^n \mathbf{V}^{n+1} \quad (27)$$

where $\mathbf{c}^n = [\mathbf{c}_e^n, \mathbf{c}_u^n]'$ is the stacked $2I \times 1$ vector.

18. If $\mathbf{V}^{n+1}$ is close enough to $\mathbf{V}^n$: stop. Otherwise, go to step 6.

Implicit Method: Matrix Representation

Alternatively, solve the linear system:

$$\mathbf{v}^{n+1} = \left(\left(\rho + \frac{1}{\Delta} \right) \mathbf{I} - \mathbf{P}^n \right)^{-1} \left[U(\mathbf{c}^n) + \frac{1}{\Delta} \mathbf{v}^n \right] \quad (28)$$

References

- Achdou, Y., J. Han, J.-M. Lasry, P.-L. Lions, and B. Moll (2022). Income and wealth distribution in macroeconomics: A continuous-time approach. *The review of economic studies* 89(1), 45–86.
- Huggett, M. (1993). The risk-free rate in heterogeneous-agent incomplete-insurance economies. *Journal of Economic Dynamics and Control* 17(5-6), 953–969.
- LeVeque, R. J. (2007). *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-dependent Problems*. Philadelphia, PA: Society for Industrial and Applied Mathematics.